AVR w czystym C – zadania

ATmega328P / Arduino Uno · lekcje 01–10

01 GPIO: wyjścia (LED)

1. ★ Migaj LED na D13 (PB5) z częstotliwością 1 Hz.
2. ★★ Biegnące światło na D2–D5. Użyj jednej pętli i przesunięcia bitowego, nie czterech kopii kodu.
3. ★★★ Licznik binarny 0–15 na 4 LED. Zmieniaj tylko bity 2–5 portu D, resztę zostaw nietkniętą (maska).
4. ★★★★ Wydziel `gpio.h` z makrami `SET_BIT`, `CLR_BIT`, `TOGGLE_BIT`, `READ_BIT` i nazwami pinów (`#define LED_PIN`). Przepisz zadania 1–3 na makra.
5. ★★★★★ Dwie sygnalizacje świetlne (czerwony/żółty/zielony) na skrzyżowaniu. Sekwencję trzymaj w tablicy struktur `{maska_LED, czas_ms}`, a program niech ją tylko przechodzi.

02 GPIO: wejścia i przyciski

1. ★ LED świeci tylko, gdy przycisk jest wciśnięty.
2. ★★ Każde wciśnięcie przełącza LED (toggle), jedno wciśnięcie = jedna zmiana. Na razie filtruj drgania przez krótkie opóźnienie.
3. ★★★ Dwa przyciski (+ / −) zmieniają licznik 0–15 pokazany binarnie na 4 LED. Licznik nie może się przekręcać.
4. ★★★★ Rozróżnij krótkie i długie wciśnięcie (próg ok. 500 ms, pomiar pętlą co 10 ms): krótkie zmienia tryb, długie zeruje licznik.
5. ★★★★★ Gra refleks: losowa LED zapala się, a gracz ma limit czasu na wciśnięcie właściwego przycisku. Punkty pokaż binarnie. Ziarno losowości weź z czasu do pierwszego wciśnięcia.

03 Timery i przerwania

1. ★ Miganie LED sterowane flagą ustawianą w ISR co 1 ms (licznik w ISR, decyzja w pętli).
2. ★★ Dwie LED migają w różnym tempie (300 ms i 700 ms) z jednego ticka, bez `delay`.
3. ★★★ Zaimplementuj `millis()` i przepisz przycisk z lekcji 2 (★★) z debouncingiem bez `delay` (stan stabilny 20 ms).
4. ★★★★ INT0 zlicza impulsy z przycisku, a pętla główna pokazuje wynik na LED. Zadbaj o atomowy odczyt licznika 16-bitowego.
5. ★★★★★ Mini-scheduler: tablica `{funkcja, okres, ostatnie_uruchomienie}` z trzema zadaniami. To zalążek Mini-RTOS.

04 UART

1. ★ Wypisz napis z PROGMEM (to Twój obecny `main.c`).
2. ★★ Wyślij liczbę dziesiętną bez `printf`, własną funkcją zamiany liczby na tekst.
3. ★★★ Echo: odebrany znak wraca do nadawcy (odbiór przez polling `RXC0`).
4. ★★★★ Komendy tekstowe `led on`, `led off`, `status`. Odbierz linię do `\r`, sparsuj ją i odpowiedz. Bufor ma się nie przepełnić.
5. ★★★★★ Odbiór przez przerwanie do **bufora kołowego** (head/tail), parser w pętli głównej, `printf` podpięty pod UART.

05 ADC

1. ★ Odczytaj potencjometr i wyślij surową wartość przez UART.
2. ★★ Przelicz na miliwolty w arytmetyce całkowitej i wypisz.
3. ★★★ Pasek postępu na 8 LED zależny od położenia potencjometru (progi).
4. ★★★★ Termometr z NTC: przeliczenie na °C (równanie beta albo tablica w PROGMEM z interpolacją), uśrednianie 8–16 próbek, alarm z histerezą.
5. ★★★★★ ADC w trybie free running z przerwaniem, bufor N ostatnich próbek, średnia krocząca i wykrywanie przekroczenia progu. Zmierz czas jednego pomiaru timerem.

06 PWM i serwo

1. ★ LED na D6 z jasnością 50%.
2. ★★ Płynne rozjaśnianie i ściemnianie ("oddychanie") sterowane tickiem z timera.
3. ★★★ Jasność LED sterowana potencjometrem (ADC → PWM), skalowanie 10 bitów do 8.
4. ★★★★ Funkcja `servo_set_angle(kąt)` mapująca 0–180° na wartość `OCR1B`. Ustaw 0°, 90°, 180°.
5. ★★★★★ Serwo sterowane potencjometrem z rampą (maksymalny krok na tick), plus melodia na buzzerze z tablicy w PROGMEM (nuty i czasy).

07 LCD HD44780 (tryb 4-bitowy)

1. ★ Zainicjuj wyświetlacz i wyślij pojedynczy znak, potem napis "Hello".
2. ★★ Zbuduj API: `lcd_cmd`, `lcd_data`, `lcd_puts`, `lcd_puts_P`, `lcd_goto(x, y)`, `lcd_clear`.
3. ★★★ Zegar sekund `MM:SS` w drugiej linii, ticki z timera i własne formatowanie z zerami wiodącymi.
4. ★★★★ Własne znaki (CGRAM): pasek postępu o 5 poziomach wypełnienia sterowany potencjometrem.
5. ★★★★★ Menu trzypoziomowe z przyciskami góra/dół/OK. Napisy w PROGMEM, maszyna stanów, odświeżaj tylko zmienione pola (bez migotania).

08 Klawiatura matrycowa 4x4

1. ★ Wykryj, czy jakikolwiek klawisz jest wciśnięty (świeci LED).
2. ★★ Zwróć znak klawisza (tablica 4x4) i wyślij go przez UART.
3. ★★★ `keypad_get()` nieblokująca: jedno wciśnięcie = jedno zdarzenie (zbocze + debouncing z `millis`).
4. ★★★★ Wpisywanie liczby do 4 cyfr: `#` zatwierdza, `*` kasuje, na LCD gwiazdki zamiast cyfr.
5. ★★★★★ Kalkulator całkowitoliczbowy: A, B, C, D jako + − × ÷, wynik na LCD, obsługa błędów (dzielenie przez 0, przepełnienie).

09 EEPROM

1. ★ Zapisz bajt, zrestartuj płytkę i odczytaj go.
2. ★★ Licznik uruchomień urządzenia zwiększany przy starcie.
3. ★★★ Zapisz i odczytaj strukturę (PIN 4 cyfry + flagi) blokowo. Z sygnaturą i wartościami domyślnymi dla pustego EEPROM.
4. ★★★★ Konfiguracja z menu LCD (np. czas blokady) zapisywana tylko przy zmianie.
5. ★★★★★ Prosty wear leveling: licznik zapisywany rotacyjnie w N komórkach ze znacznikiem najnowszego wpisu, plus CRC8 wykrywające uszkodzenie danych.

10 I2C i SPI

1. ★ 74HC595 przez SPI: wyślij bajt i pokaż wzór na 8 LED.
2. ★★ 74HC595: animacja biegnącego światła sterowana timerem.
3. ★★★ I2C od zera: `i2c_start`, `i2c_stop`, `i2c_write`, `i2c_read`. Skaner adresów wypisujący znalezione urządzenia przez UART.
4. ★★★★ DS1307: odczyt czasu (BCD → liczby), wyświetlanie na LCD i ustawianie godziny przyciskami.
5. ★★★★★ Wybierz jedno: zegar na matrycy MAX7219 (SPI) albo własny sterownik tekstu dla SSD1306 z czcionką w PROGMEM.